



بسمه تعالی

گام‌های تکمیل آزمایشگاه مخابرات دیجیتال

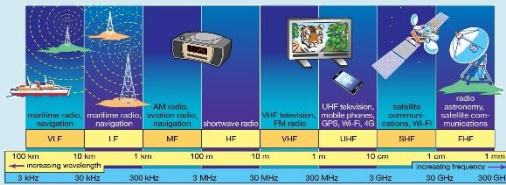
استاد: دکتر علی الفت

ویرایش ۱/۵ (آزمایشی)

فهرست مطالب آزمایشگاه مخابرات دیجیتال

آزمایش اول: رادیونرم افزار و پایش طیف سیگنال های رادیویی	۳
آزمایش دوم: معرفی مدولاسیون دیجیتال خطی	۵
آزمایش سوم: کاوش در مدولاسیون دیجیتال خطی	۸
آزمایش چهارم: مدولاسیون FSK همدوس	۱۹
آزمایش پنجم: آشکارسازی ناهمدوس	۲۹
آزمایش ششم: انتقال دیجیتال از درون کانال باند محدود AWGN	۳۳





آزمایش اول

رادیونرم افزار و پایش سیگنال های رادیویی

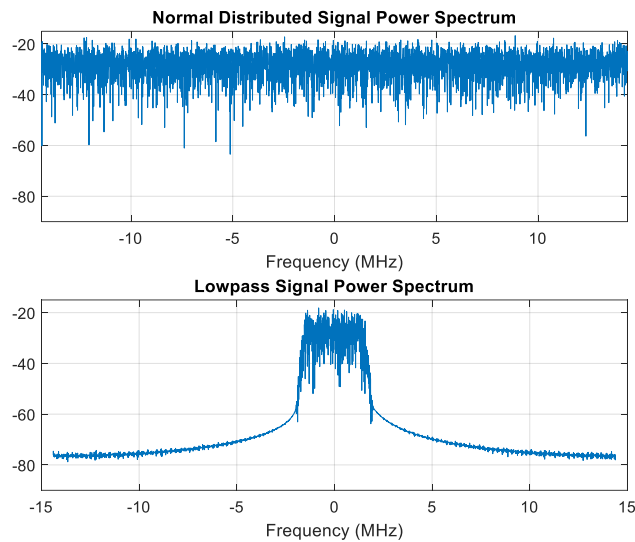
شرح آزمایش



آزمایش ۱-۱: معادل پایین گذر و میان گذر سیگنال

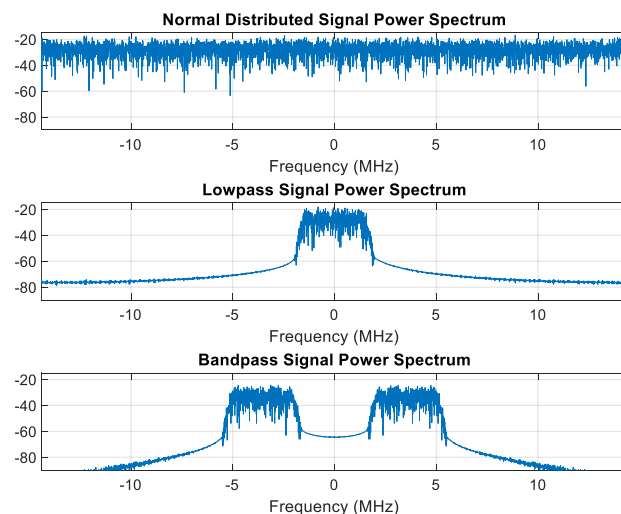
۱. تولید فرآیند میان گذر و پایین گذر:

أ. تولید فرآیند پایین گذر:



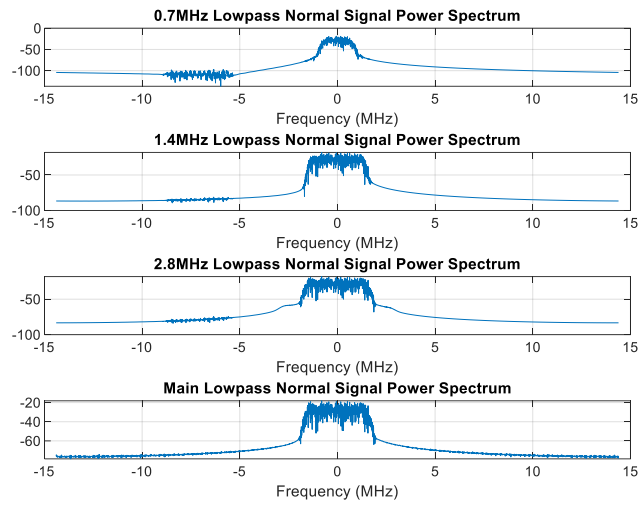
```
x_power = 1.0122
x_baseband_power = 0.1139
bw_noise = 3.3081e6
```

ب. تولید فرآیند میان گذر معادل یک فرآیند پایین گذر:

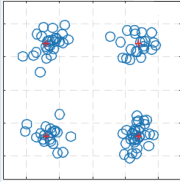


```
x_power = 1.0122
x_baseband_power = 0.1139
x_bandpass_power = 0.0570
```

ت. تولید فرآیند پایین گذر از یک سیگنال میان گذر:



```
x_baseband_power = 0.1139
x_baseband_0p7_power = 0.0455
x_baseband_1p4_power = 0.0954
x_baseband_2p8_power = 0.1141
```



آزمایش دوم

معرفی مدولاسیون دیجیتال خطی

شرح آزمایش



آزمایش ۱-۲: پیاده‌سازی فرستنده مدولاسیون PAM

محاسبات و خروجی‌های زیر بر اساس این است که ابتدای کد خود (1) rng گذاشته باشید.

گام ۱. تولید بیت: می‌بایست ۲۰ عدد در قالب ماتریس زیر تولید شود.

```
txBit =
    0    0
    1    1
    0    0
    0    1
    0    0
    0    1
    0    0
    0    1
    0    0
    0    0
    1    0
```

گام ۲. نگاشت بیت به سمبل:

گام ۱. تولید دنباله‌ی کد گری به صورت باینری: می‌بایست تابعی به صورت زیر داشته باشید.

```
function [grayMatrix] = grayMatrixGenerator(k)
```

گام ۲. کدگذاری گری بیت‌های تولیدی:

```
grayMatrix =
    0    0
    0    1
    1    1
    1    0
```

گام ۳. تولید سمبل‌های ارسالی:

```
symbolIndex =
    0
    2
    0
    1
    0
    1
    0
    1
    0
    0
    3
```

```
symbolArray =
-1.3416
 0.4472
-1.3416
-0.4472
-1.3416
-0.4472
-1.3416
-0.4472
-1.3416
```

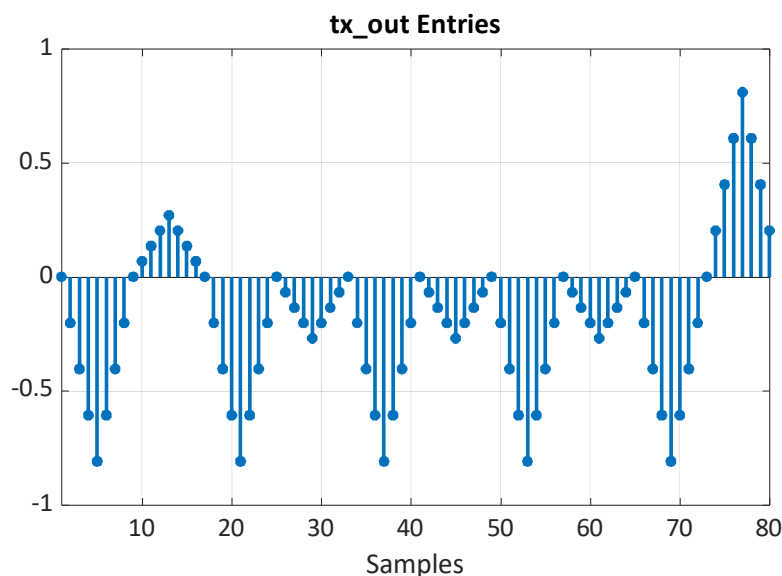
-0.4472
-1.3416
1.3416

گام ۳. شکل‌دهی پالس ارسالی: ابتدا می‌بایست تابعی به صورت زیر داشت.

```
function [txSamples, cons] = pulseModulation(symbolIndex, modulation, M, fs,
nSymbolSamples, pulseName, pulseShapingMode, varargin)
```

عددهای زیر به ترتیب در سطر درایه‌های بردار txSamples می‌باشد.

0.0000	-0.2023	-0.4045	-0.6068	-0.8090	-0.6068	-0.4045
-0.2023	0.0000	0.0674	0.1348	0.2023	0.2697	0.2023
0.1348	0.0674	0.0000	-0.2023	-0.4045	-0.6068	-0.8090
-0.6068	-0.4045	-0.2023	0.0000	-0.0674	-0.1348	-0.2023
-0.2697	-0.2023	-0.1348	-0.0674	0.0000	-0.2023	-0.4045
-0.6068	-0.8090	-0.6068	-0.4045	-0.2023	0.0000	-0.0674
-0.1348	-0.2023	-0.2697	-0.2023	-0.1348	-0.0674	0.0000
-0.2023	-0.4045	-0.6068	-0.8090	-0.6068	-0.4045	-0.2023
0.0000	-0.0674	-0.1348	-0.2023	-0.2697	-0.2023	-0.1348
-0.0674	0.0000	-0.2023	-0.4045	-0.6068	-0.8090	-0.6068
-0.4045	-0.2023	0.0000	0.2023	0.4045	0.6068	0.8090
0.6068	0.4045	0.2023				



آزمایش ۲-۲: پیاده‌سازی گیرنده‌ی مدولاسیون PAM

۱. تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه: می‌بایست تابعی به صورت زیر در یک فایل جداگانه داشته باشید.

```
function [rxSymbols] = corr_match(rxSamples, p, nSymbolSamples, rxMode)
```

۲. گیرنده‌ی حداقل فاصله: می‌بایست تابعی به صورت زیر در یک فایل جداگانه داشته باشید.

```
function [detectedSymbols] = minDistanceDetector(rxSymbols, constellation);
```

۳. دمدولاسیون یا آشکارسازی پالس: می‌بایست تابعی به صورت زیر در یک فایل جداگانه داشته باشید.

```
function [detectedSymbolsIndex, rxSymbols] = pulseDemodulation(rxSamples,
modulation, M, fs, nSymbolSamples, pulseName, rxMode, varargin)
```

```
detectedSymbolsIndex =
1
3
1
```

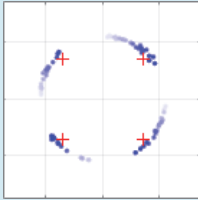
2
1
2
1
2
1
4

```
rxSymbols =
-1.3416
 0.4472
-1.3416
-0.4472
-1.3416
-0.4472
-1.3416
-0.4472
-1.3416
 1.3416
```

```
detectedBitsDecimal =
0
3
0
1
0
1
0
1
0
1
2
```

```
detectedBits =
0 0
1 1
0 0
0 1
0 0
0 1
0 0
0 1
0 0
1 0
```

.



آزمایش سوم

کاوش در مدولاسیون دیجیتال خطی

شرح آزمایش

ابتدای فایل شبیه‌سازی از `rng(1)` استفاده نمایید.

آزمایش ۲-۳: پیاده‌سازی فرستنده مدولاسیون

فرض می‌کنیم `pkt_size` برابر با ۱۰ باشد و مدولاسیون 4PSK است.

گام ۱. تولید بیت: صحت‌سنجی ماتریس `txBit` با اندازه‌ی `pkt_size × k` می‌باشد.

`txBit =`

```
0 0
1 1
0 0
0 1
0 0
0 1
0 0
0 1
0 0
0 0
1 0
```

گام ۲. نگاشت بیت به سمبل:

کدگذاری گری بیت‌های تولیدی:

صحت‌سنجی ماتریس `grayMatrix`

`grayMatrix =`

```
0 0
0 1
1 1
1 0
```

صحت‌سنجی بردار `symbolIndex`

`symbolIndex =`

```
0
2
0
1
0
1
0
1
0
3
```

کدگذاری طبیعی بیت‌های تولیدی:

صحت‌سنجی بردار `symbolIndex` با کدگذاری باینری


```
symbolIndex =
```

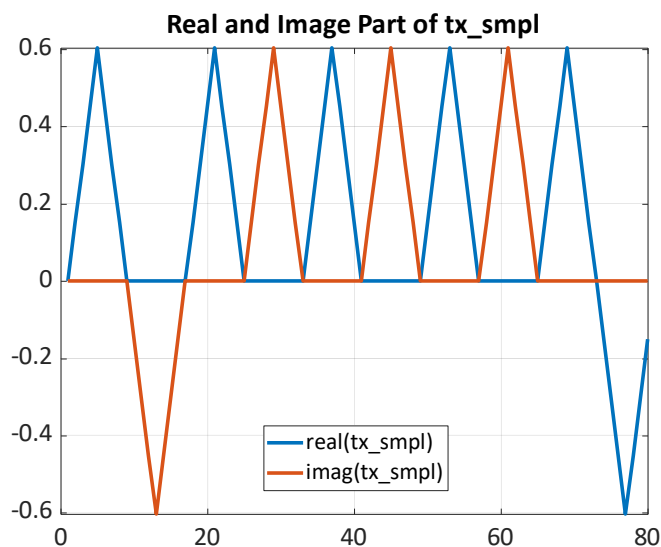
```
0
3
0
1
0
1
0
1
0
2
```

تولید سمبل‌های ارسالی:

صحت‌سنجی بردار `symbolArray`

```
symbolArray =
```

```
1.0000 + 0.0000i
-0.0000 - 1.0000i
1.0000 + 0.0000i
0.0000 + 1.0000i
1.0000 + 0.0000i
0.0000 + 1.0000i
1.0000 + 0.0000i
0.0000 + 1.0000i
1.0000 + 0.0000i
-1.0000 + 0.0000i
```

گام ۳. شکل‌دهی پالس ارسالی: صحت‌سنجی بردار `txSamples`

گام ۴. شبیه‌سازی کانال با نویز سفید گاوسی

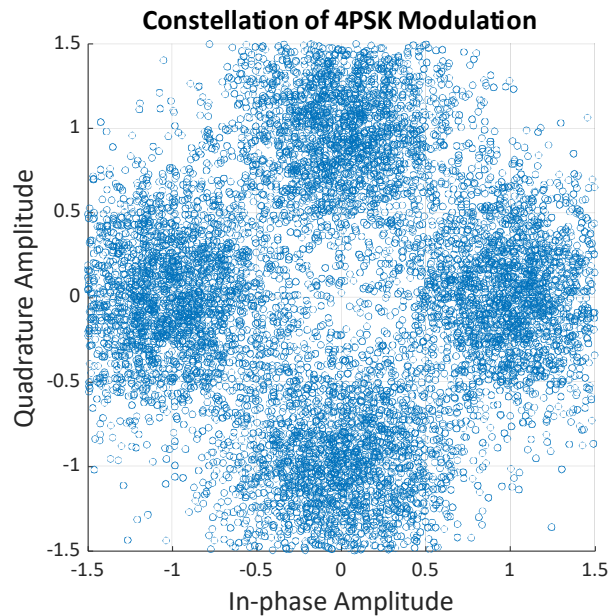
افزودن نویز به سیگنال: صحت‌سنجی بخشی از بردار `rxSamples_noise`

```
rxSamples_noise =
```

```
0.4160 - 0.1504i
0.8690 + 0.5969i
0.2041 + 0.1294i
0.6659 - 0.3883i
1.2335 + 0.6832i
1.0801 + 0.2205i
-0.3587 + 0.2327i
-0.2213 - 0.5180i
```

-0.1477 + 0.3023i
 0.4963 + 0.0547i
 -0.4841 - 0.6267i
 -0.1035 - 0.1709i
 ...

گام ۵. دمدولاسیون یا آشکارسازی پالس: صحت‌سنجی سمبل‌های دریافتی (`rxSymbols`) با استفاده از نمودار. در این جا تعداد سمبل‌ها برابر ۱۰۰۰۰ در نظر گرفته می‌شود.



گام ۶. محاسبه‌ی خطای سمبل و خطای بیت:
 BER و SER با اعمال کدگذاری گری:

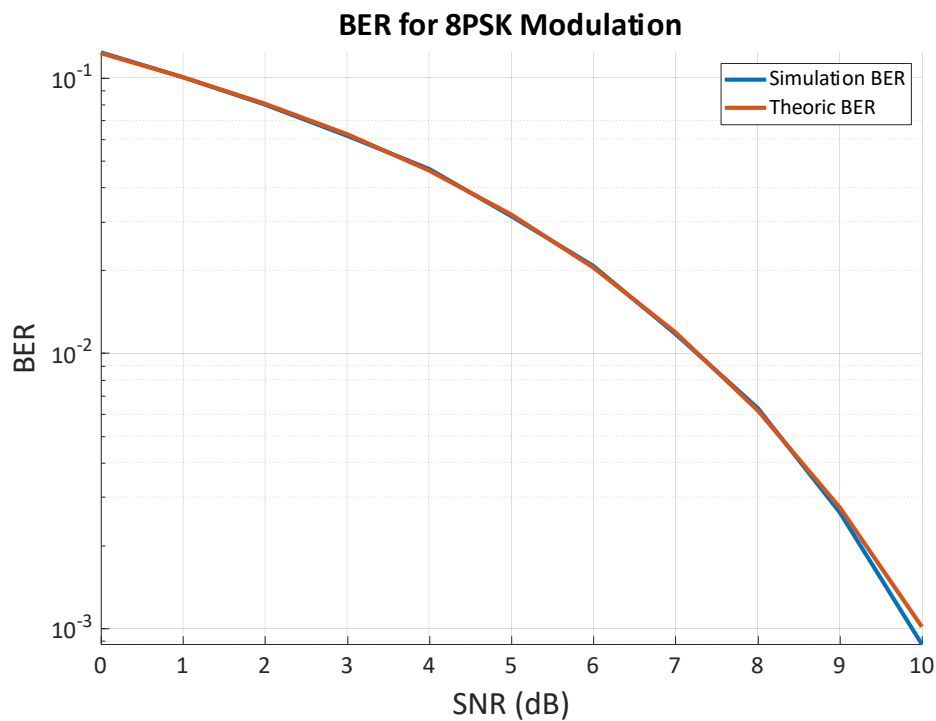
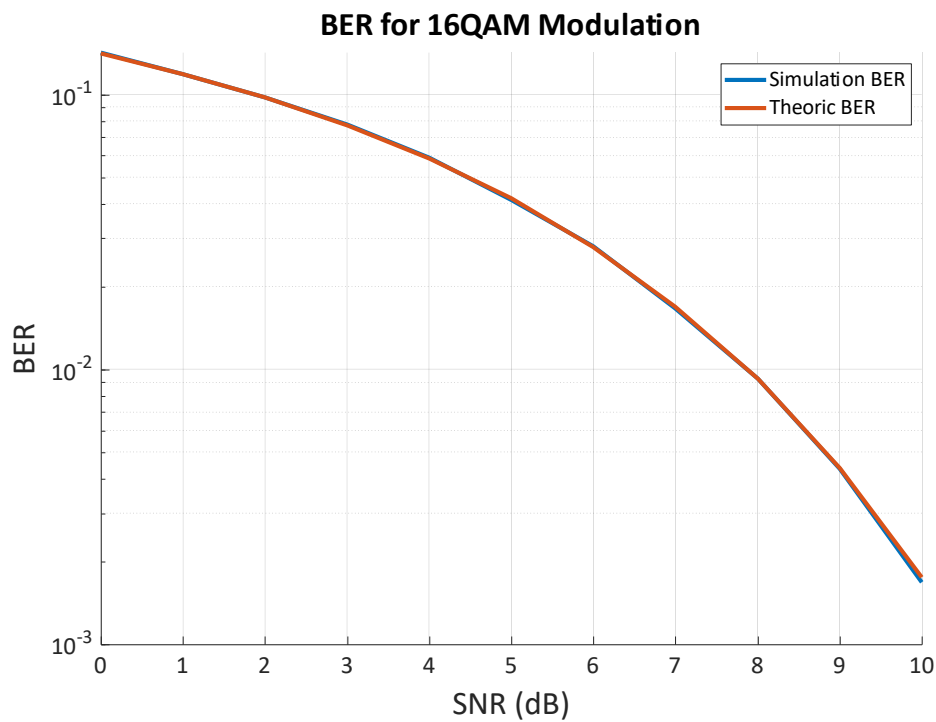
SER: 0.0454 BER: 0.0228

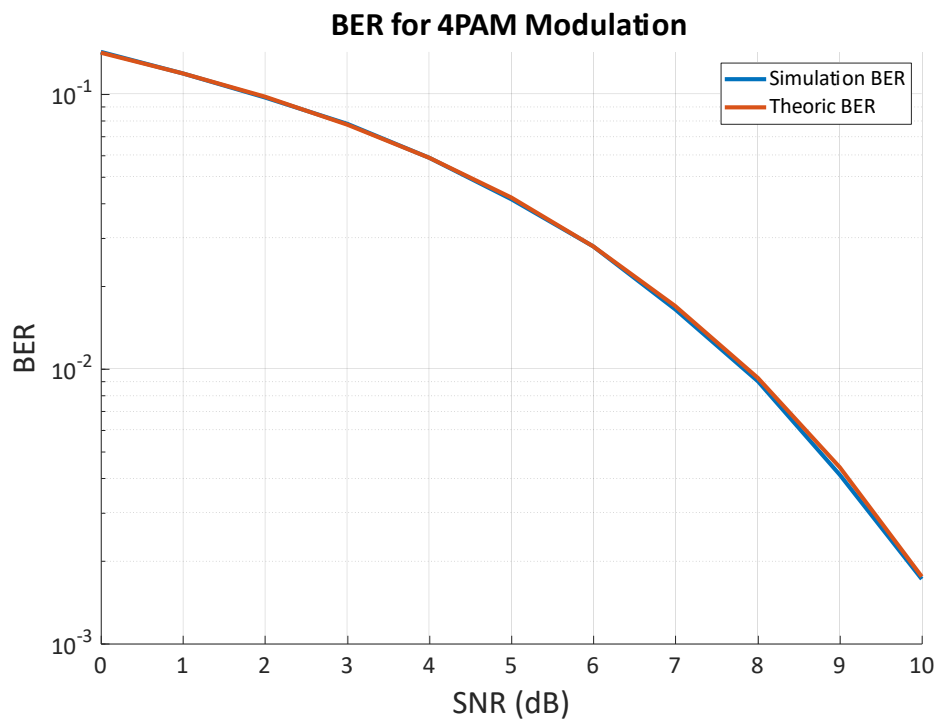
BER و SER بدون اعمال کدگذاری گری:

SER: 0.0447 BER: 0.0336

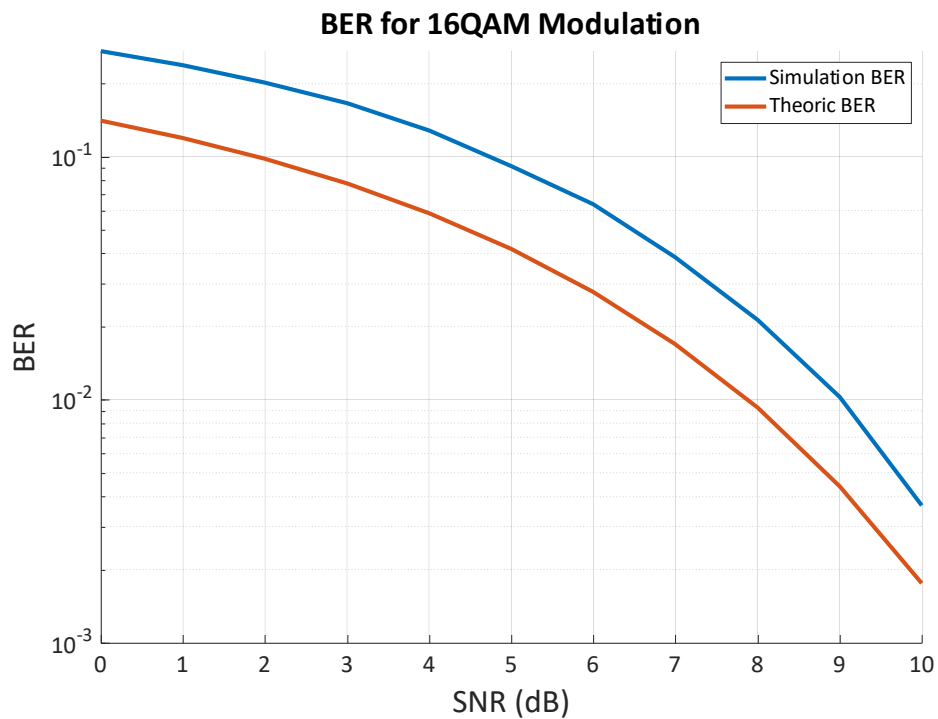
آزمایش ۵-۳: خواسته‌های کلی

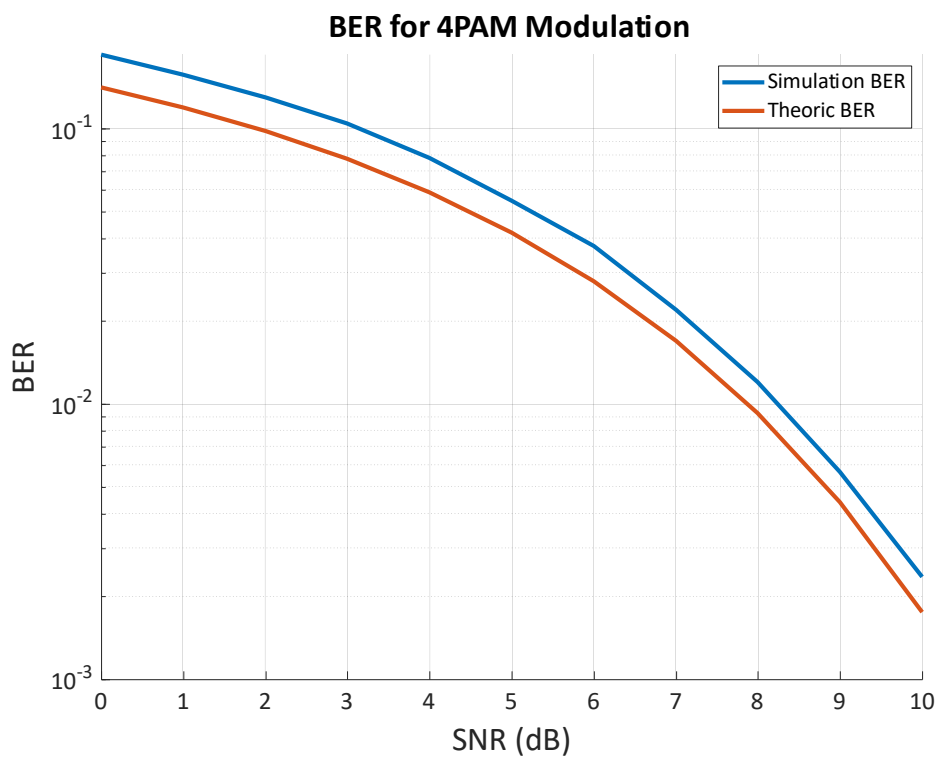
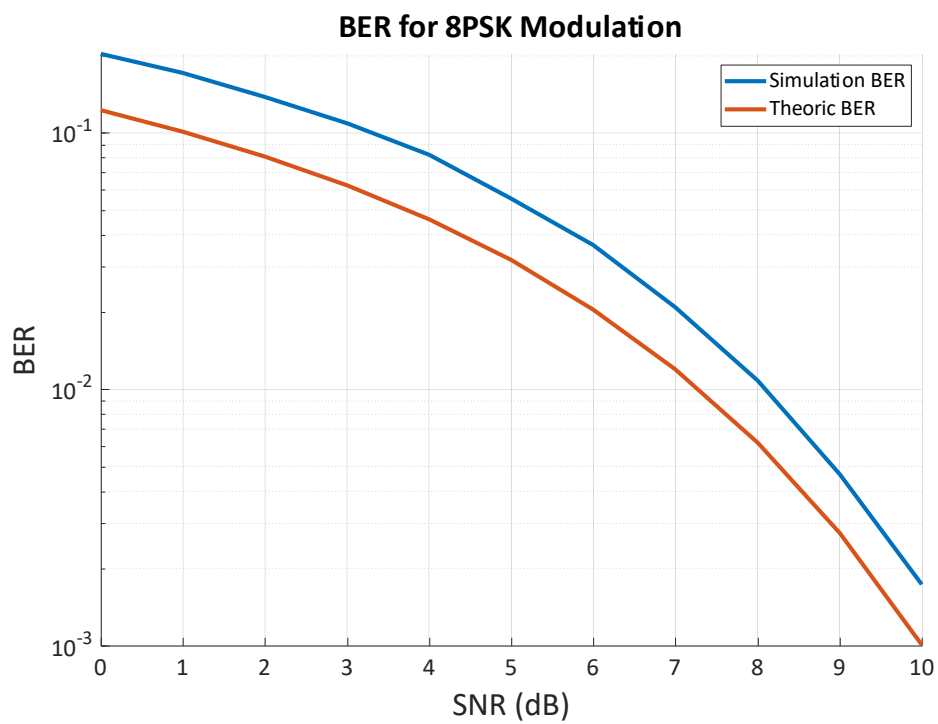
در نمودارهایی که در ادامه آمده است ابتدای برنامه $\text{rng}(1)$ می‌باشد و تعداد سمبل‌ها ۱۰۰۰۰۰ می‌باشد. منظومه‌های سیگنالی نیز در نسبت سیگنال به نویز ۱۰dB رسم شده است. گام ۷. رسم نمودار نرخ خطای بیت با اعمال کدگذاری گری

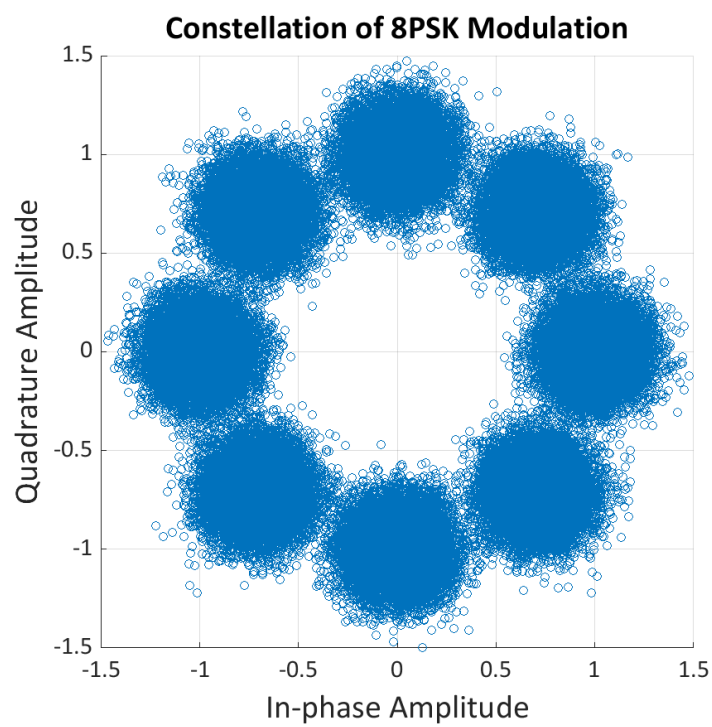
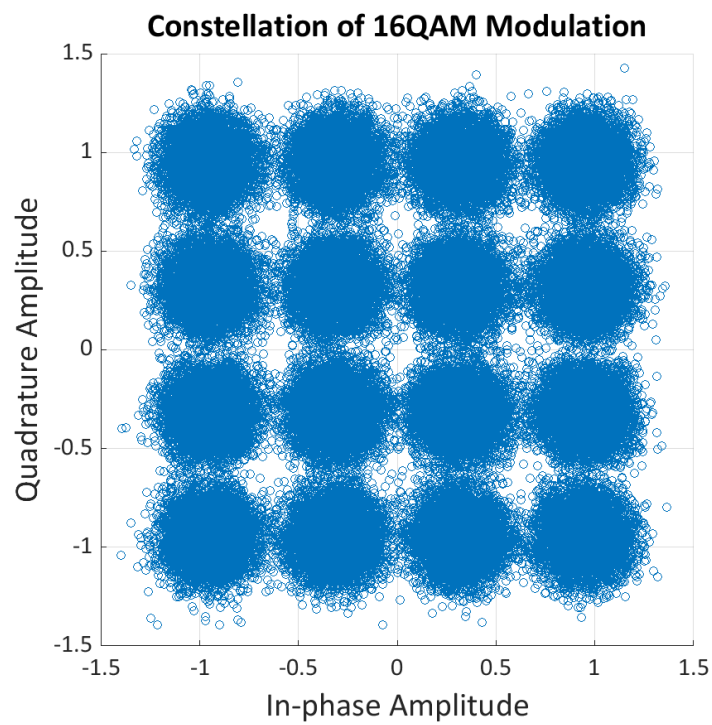


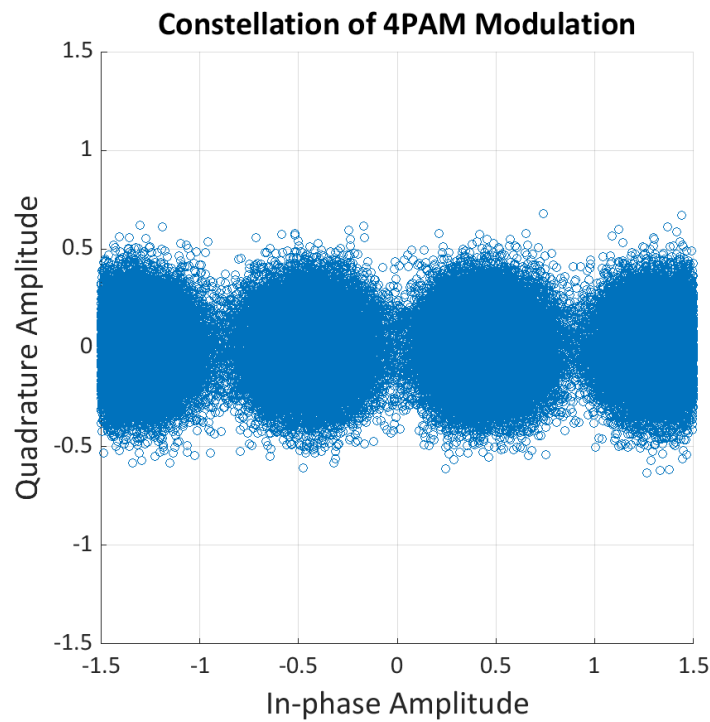


گام ۸. نمودارهای نرخ خطا بدون اعمال کدگذاری گری





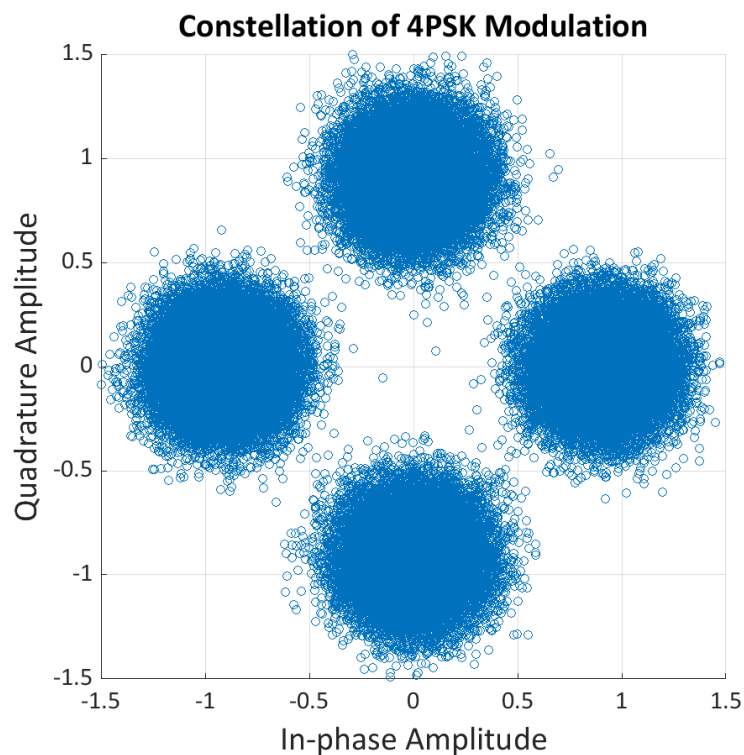




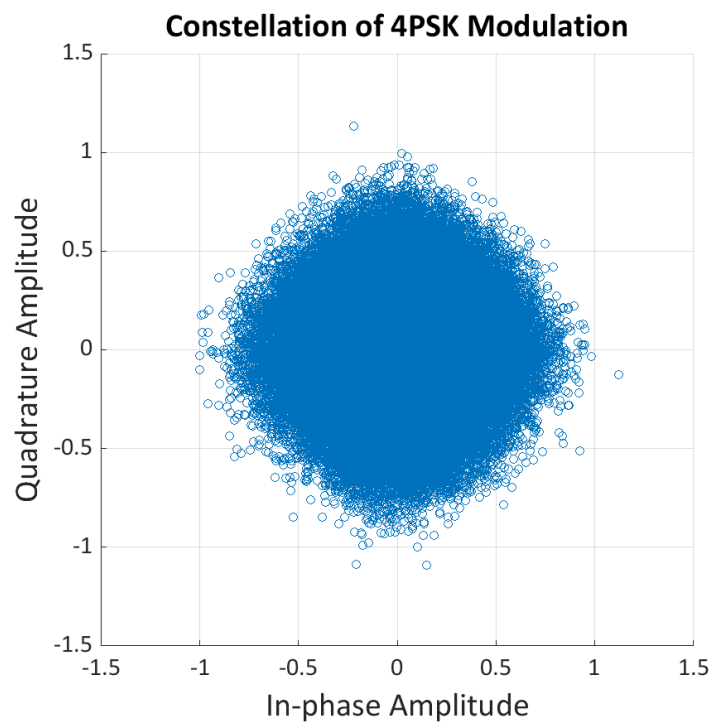
گام ۱۰. صحت‌سنجی یکسانی نتایج با دو شکل موج مثلثی و root raised cosine

گام ۱۱. اثر تأخیر در کانال:

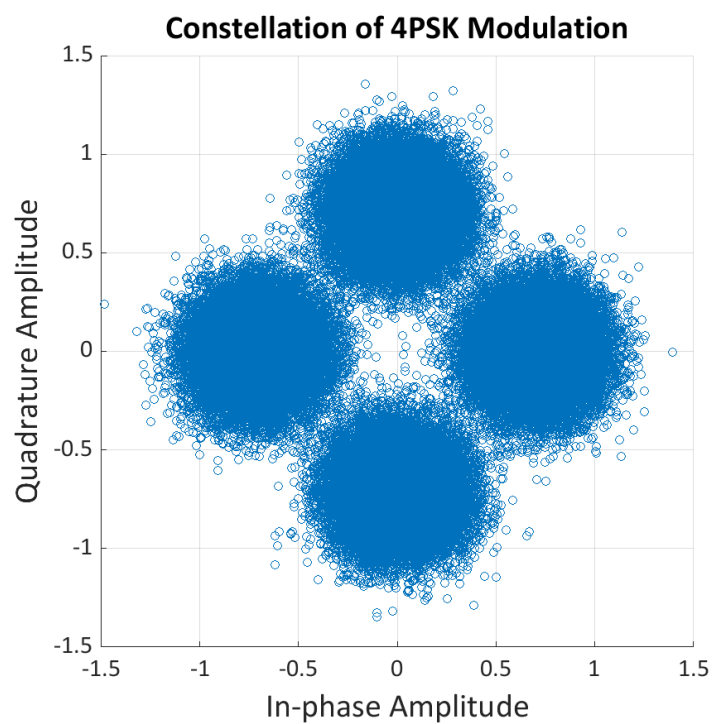
نمودارها و نرخ خطای بیت به ترتیب برای تأخیر کانال برابر با گرد شده‌ی 0.1، 0.5 و 0.8 برابر طول یک سمبل می‌باشد. مدولاسیون 4PSK و نسبت سیگنال به نویز (E_b/N_0) برابر با 10dB است



BER: 2.5×10^{-5}

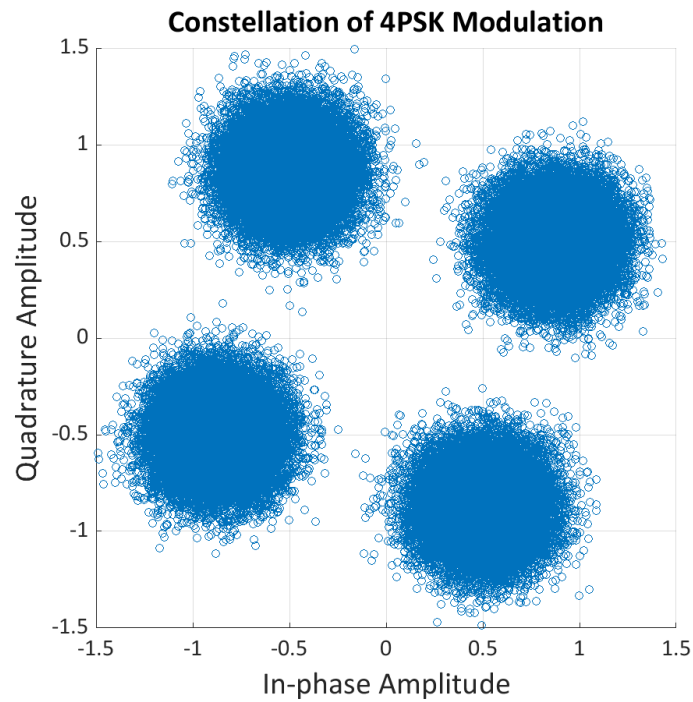


BER : 0.3230



BER : 0.5019

گام ۱۲. اثر فاز اضافی کانال بر نرخ خطای بیت:

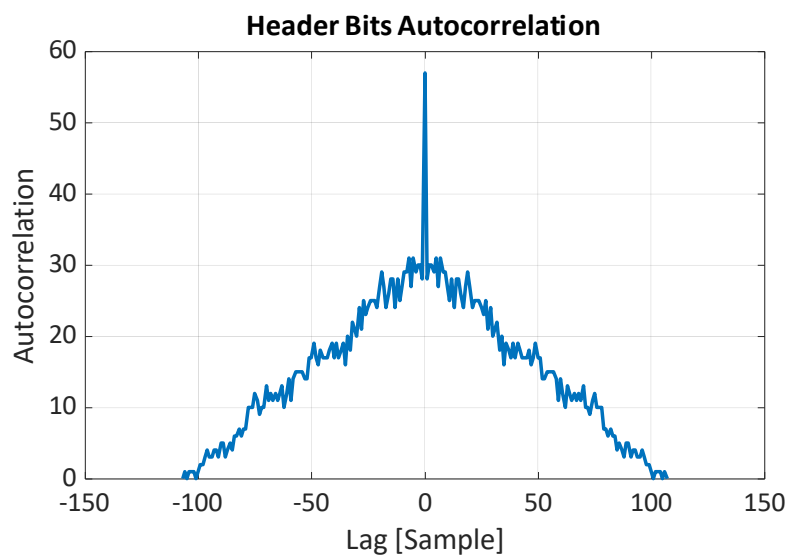


BER : 0.0382

گام ۱۳. اضافه کردن هدر

ستون اول ماتریس `hdr_bit` و خودهمبستگی آن

1	0	0	0	0	0	1	1	0	1	1	0	1
1	0	0	0	0	0	1	1	1	1	0	1	1
1	1	1	1	1	1	1	0	1	0	1	0	1
1	0	1	0	1	0	0	1	0	1	1	1	1
0	1	0	1	1	1	0	0	0	1	0	0	1
0	1	0	0	0	1	1	0	0	0	1	1	0
0	1	1	0	1	0	1	0	0	1	1	0	1
0	0	0	0	1	1	0	1	1	0	0	1	1
1	0	1	0									



مقدار قله‌ی خودهمبستگی نمونه‌های زمانی هدر: 108 با فاز صفر

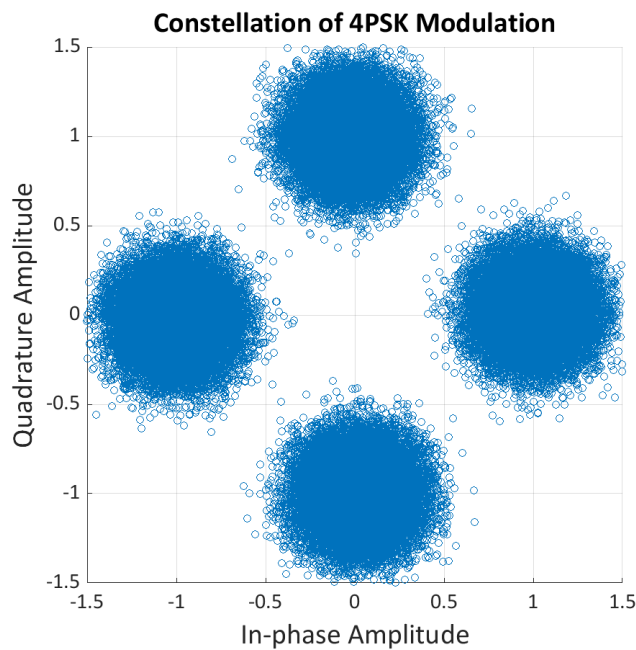
`hdr_corr_max =`

1.0800e+02

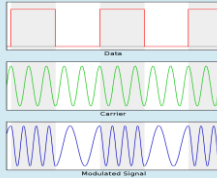
خروجی همبستگی متقابل بین نمونه‌های هدر و نمونه‌های سیگنال دریافتی:

```
corr_out_rx_max =
    96.8562 +52.5040i
idx_max =
    5
```

گام ۱۴. جبران‌سازی فاز و تأخیر
منظومه‌ی سیگنالی بعد از جبران‌سازی



BER و SER در SNR برابر با 10dB با تعداد نمونه‌های ۱۰۰۰۰۰ صفر می‌شود.



آزمایش چهارم

مدولاسیون FSK همدوس

گام‌های تکمیل آزمایش



ابتدای فایل شبیه‌سازی از `rng(1)` استفاده نمایید.

آزمایش ۲-۴: پیاده‌سازی فرستنده مدولاسیون FSK

فرض می‌کنیم `pkt_size` برابر با ۱۰ باشد.

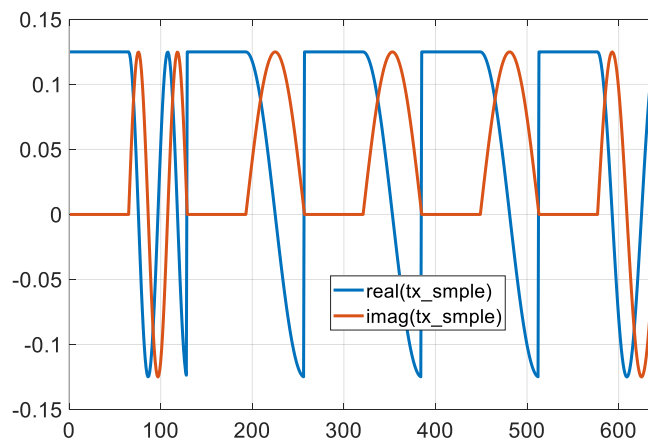
گام ۱. تولید بیت: صحت‌سنجی ماتریس `txBit`:

```
txBit =
    0    0
    1    1
    0    0
    0    1
    0    0
    0    1
    0    0
    0    1
    0    0
    0    1
    0    0
    1    0
```

گام ۲. نگاشت بیت به شماره‌ی سمبل: صحت‌سنجی بردار `symbolIndex`

```
symbolIndex =
    0
    3
    0
    1
    0
    1
    0
    1
    0
    1
    0
    2
```

گام ۳. تولید نمونه‌های زمانی سمبل ارسالی: صحت‌سنجی تولید نمونه‌های ارسالی `txSamples`



آزمایش ۳-۴: مدل سازی کانال

گام ۴. شبیه سازی کانال با نویز سفید گاوسی: صحت سنجی بردار نویز تولیدی برای E_b/N_0 برابر با 10dB (بخشی از بردار نویز تولیدی)

```
noise_smp1 =
0.1858 + 0.1383i
0.3208 + 0.4851i
-0.0435 - 0.0979i
0.0954 + 0.1198i
0.2816 + 0.1626i
0.2804 + 0.0084i
-0.2949 + 0.0333i
-0.1662 + 0.1822i
-0.0660 - 0.0557i
0.2217 + 0.0815i
-0.2163 + 0.1521i
-0.0463 + 0.0612i
0.2009 - 0.0356i
0.0104 + 0.0394i
0.0714 - 0.3029i
-0.0509 + 0.1125i
0.1247 + 0.2471i
0.1468 - 0.1305i
-0.0776 - 0.0898i
0.2842 - 0.1144i
0.0934 - 0.1595i
```

آزمایش ۴-۴: پیاده سازی گیرنده ی مدولاسیون FSK

گام ۵. دمدولاسیون یا آشکارسازی پالس: صحت سنجی مراحل میانی دمدولاسیون

ماتریس خروجی کرولیتورهای چهارگانه:

1.0071 + 0.2678i	0.2301 - 0.5272i	0.1354 + 0.0768i	0.1316 - 0.2025i
-0.0779 + 0.3464i	0.1913 + 0.0275i	-0.1206 + 0.6427i	1.1417 + 0.1540i
1.0595 - 0.0011i	-0.0914 - 0.8211i	-0.1949 + 0.2063i	0.2696 - 0.1893i
0.3165 + 0.4536i	0.9049 - 0.1232i	0.1223 - 0.8286i	-0.2220 - 0.0933i
0.9524 - 0.1912i	-0.0836 - 0.5960i	0.1236 - 0.1153i	-0.0791 - 0.3385i
-0.0162 + 0.6201i	1.1771 - 0.0727i	-0.0886 - 0.8867i	-0.0930 + 0.0668i
1.0855 - 0.0804i	-0.1852 - 0.5615i	0.0401 + 0.1100i	-0.1268 - 0.1138i
-0.0933 + 0.7522i	1.0438 + 0.0047i	-0.1195 - 0.4640i	0.2294 + 0.1137i
0.8772 + 0.2424i	0.1505 - 0.5703i	-0.1465 - 0.0123i	-0.0230 + 0.0648i
-0.1540 + 0.0173i	0.1698 + 0.8738i	1.1034 - 0.3322i	-0.3168 - 0.5730i

detectedSymbolsIndex =

```
1
4
1
2
1
2
1
2
1
3
```

detectedSymbolsIndex بسته به نحوه ی کد نویسی می تواند بردار فوق منهای ۱ باشد.

rxSymbols =

1.0071	0.2301	0.1354	0.1316
-0.0779	0.1913	-0.1206	1.1417
1.0595	-0.0914	-0.1949	0.2696
0.3165	0.9049	0.1223	-0.2220
0.9524	-0.0836	0.1236	-0.0791

-0.0162	1.1771	-0.0886	-0.0930
1.0855	-0.1852	0.0401	-0.1268
-0.0933	1.0438	-0.1195	0.2294
0.8772	0.1505	-0.1465	-0.0230
-0.1540	0.1698	1.1034	-0.3168

`rxSymbols` در واقع خروجی ای است که می توان از آن برای رسم منظومه مدولاسیون 4FSK در چهار بعد استفاده نمود.

گام ۶. محاسبه ی خطای سمبل: صحت سنجی خطای سمبل با `pkt_size` برابر با 10^5

`ser =`

`1.0000e-05`

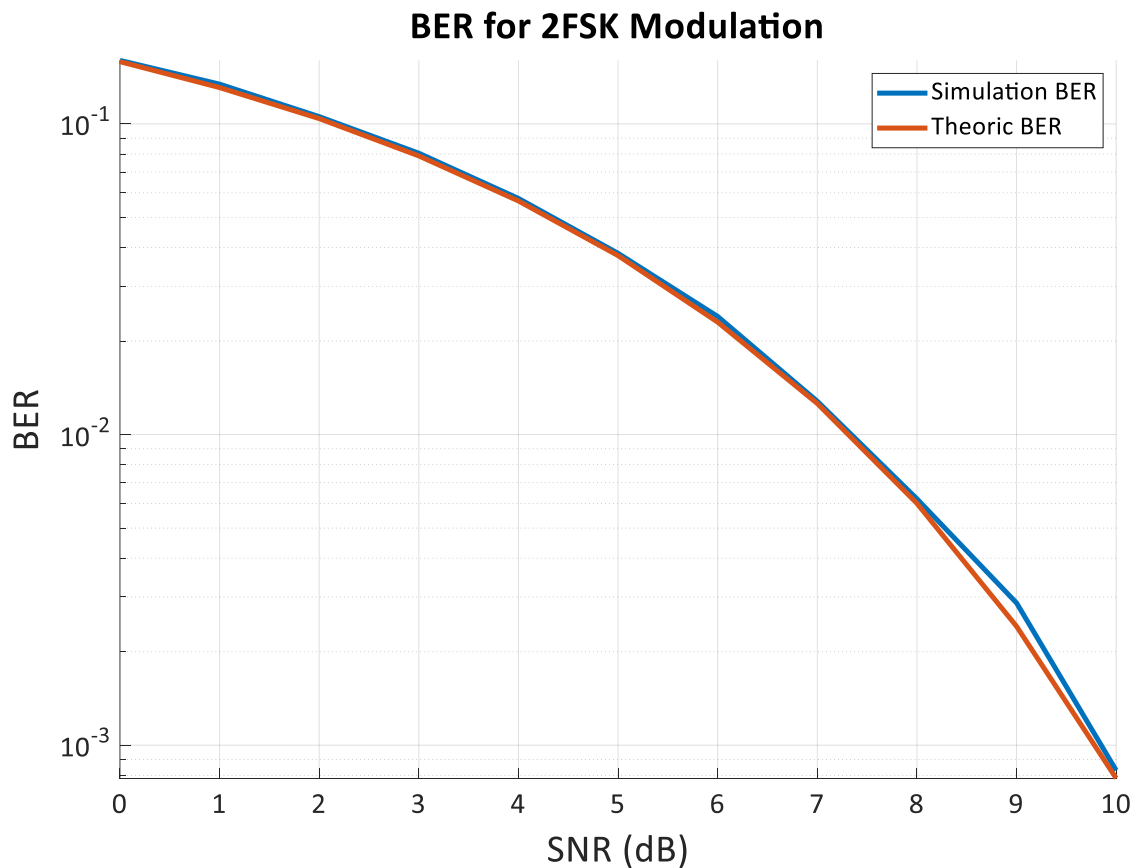
گام ۷. تبدیل سمبل به بیت و محاسبه ی خطای بیت: صحت سنجی خطای بیت با `pkt_size` برابر با 10^5

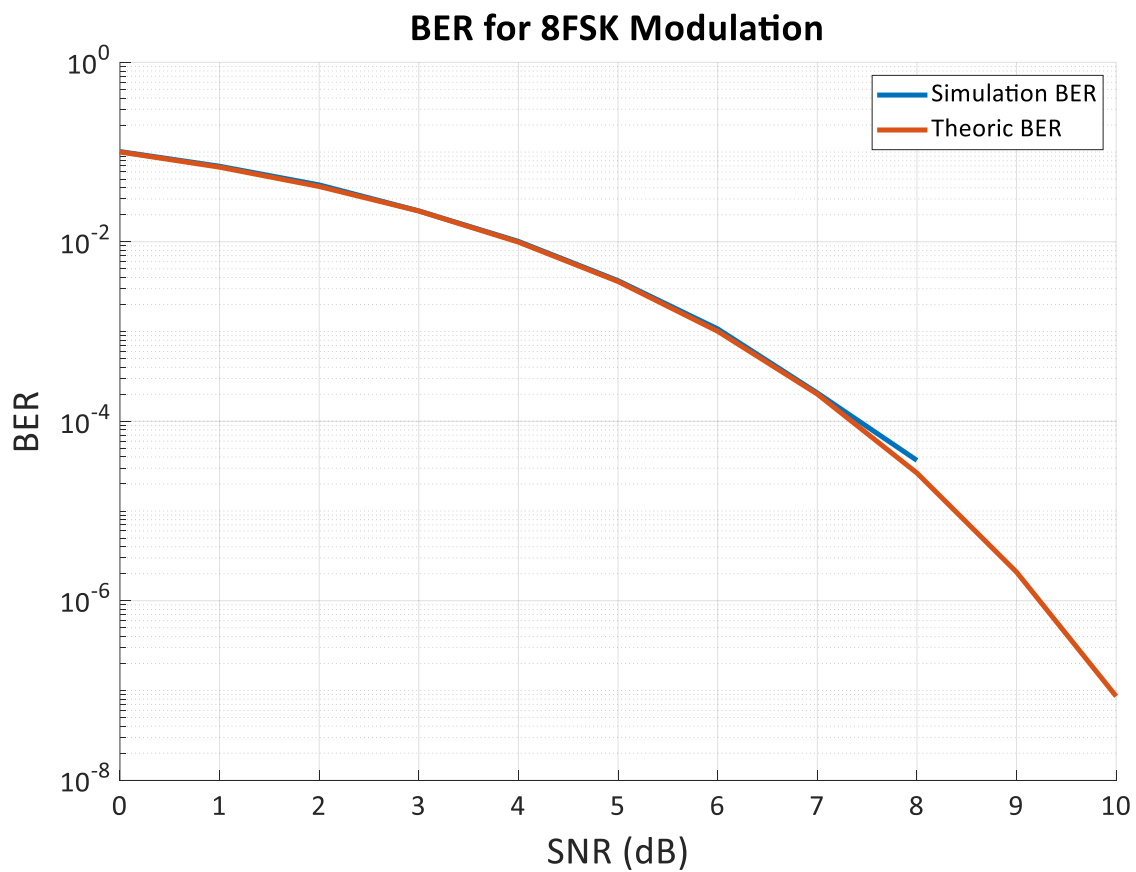
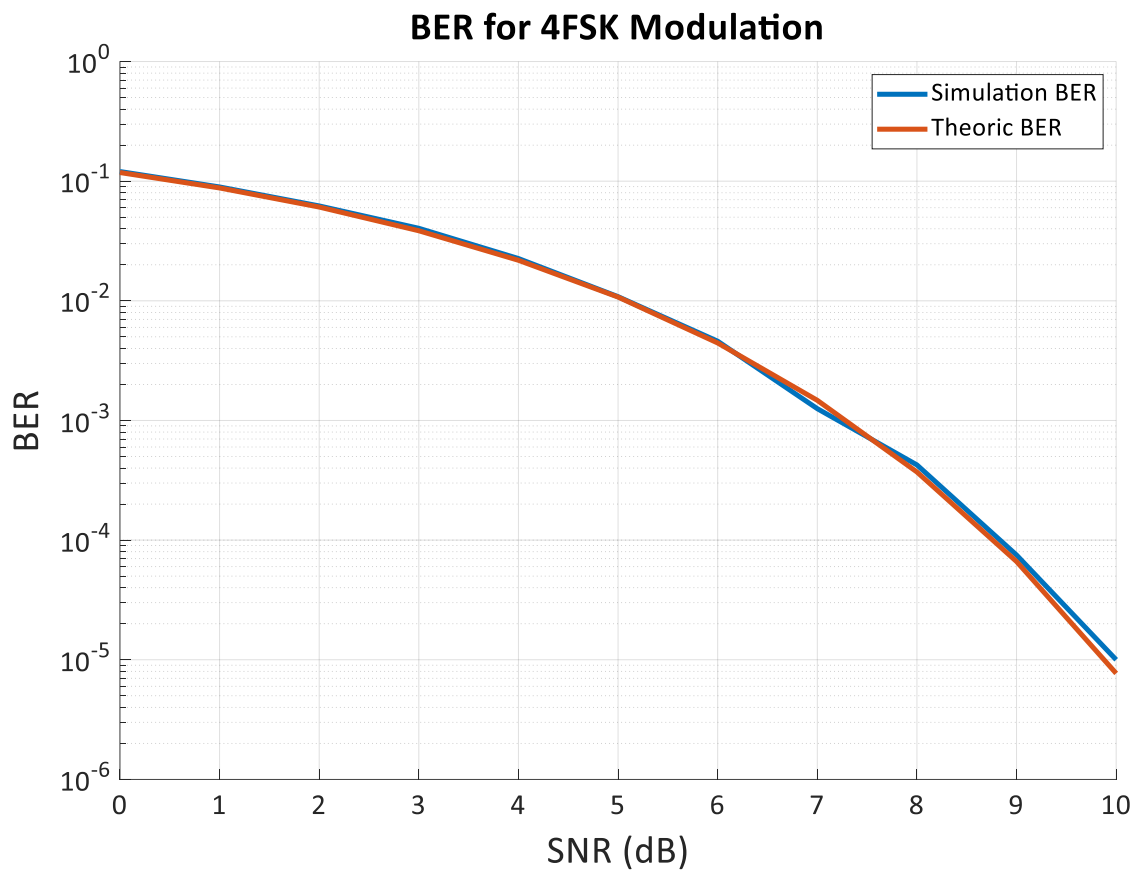
`ber =`

`5.0000e-06`

آزمایش ۵-۴: خواسته های کلی

گام ۸. رسم نمودار نرخ خطای بیت: صحت سنجی نمودار خطای بیت با `pkt_size` برابر با 10^5

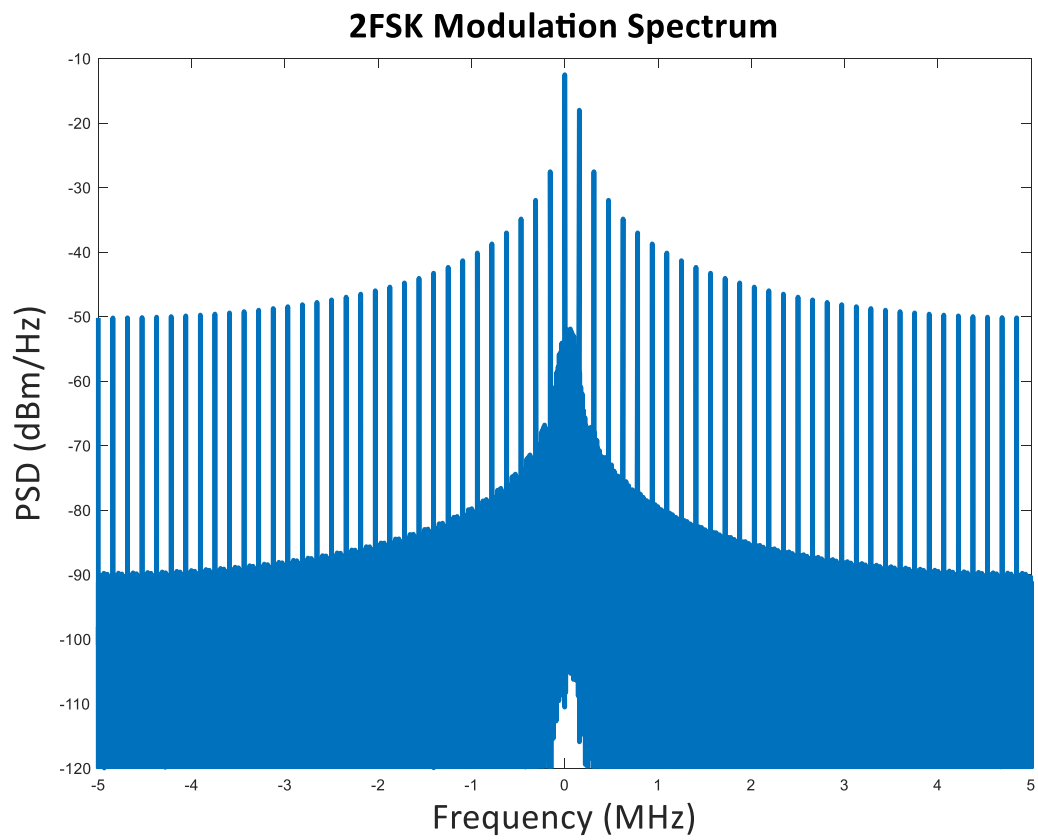




آزمایش ۴-۵: خواسته‌های کلی

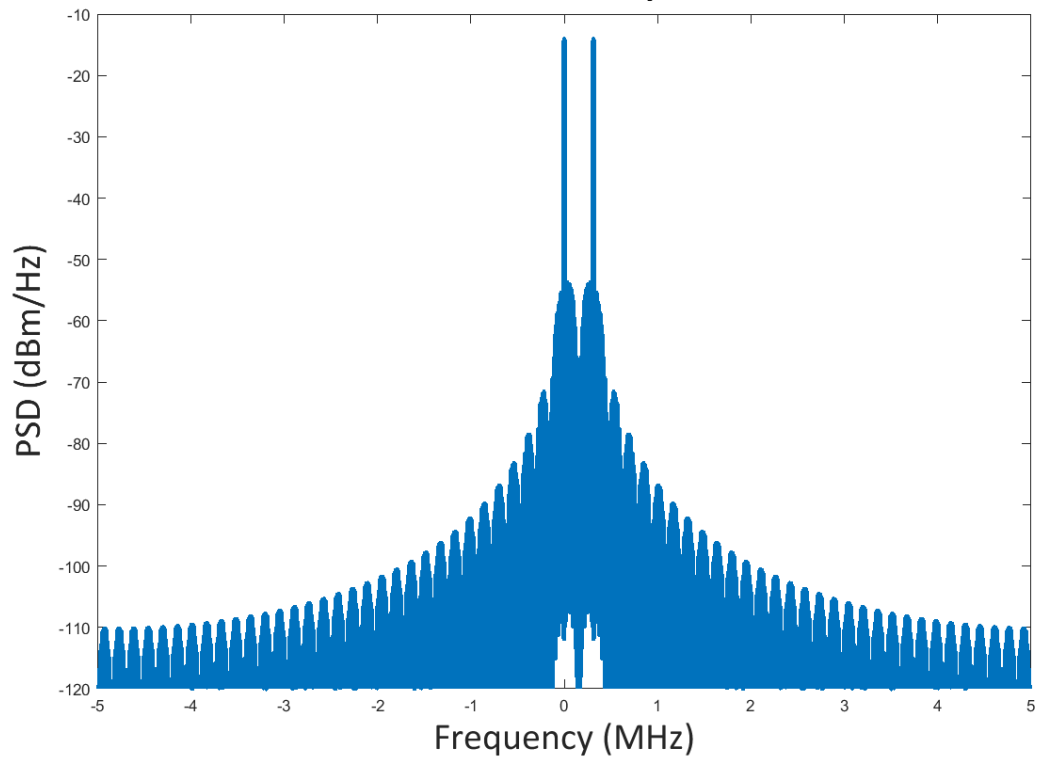
گام ۹. اثر Δf بر روی پهنای باند ارسالی:

$$\Delta f = \frac{1}{2T_s}$$



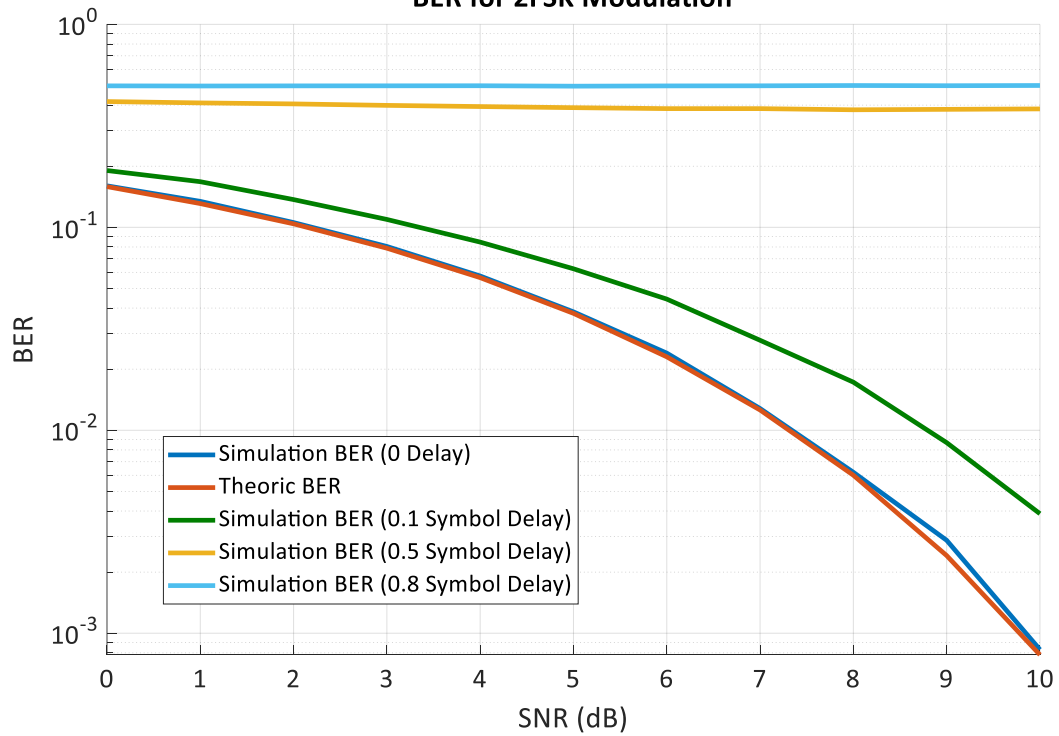
$$\Delta f = \frac{2}{T_s}$$

2FSK Modulation Spectrum

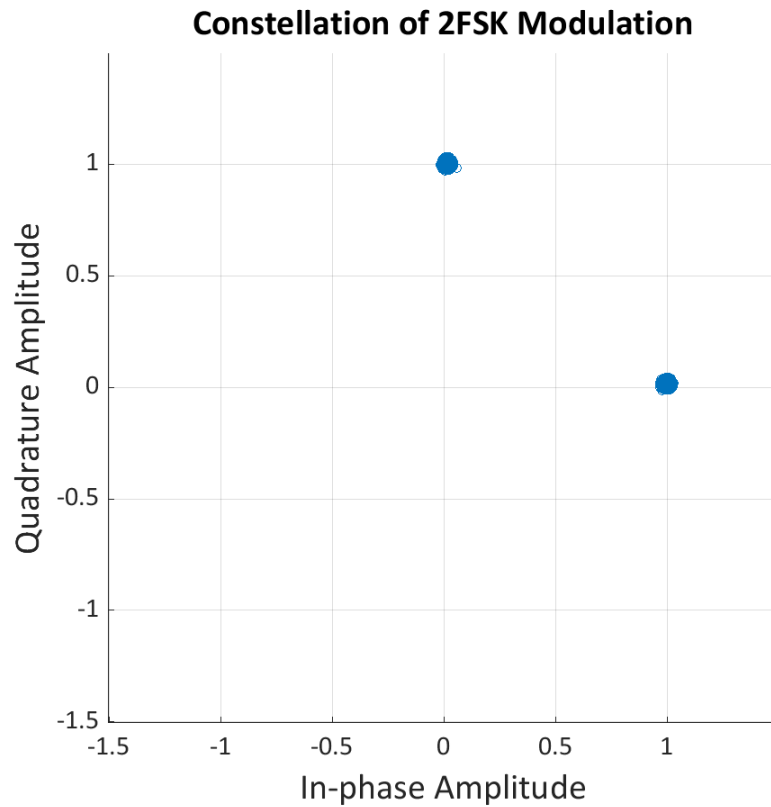


گام ۱۰. اثر تأخیر در کانال بر روی رسم نمودار نرخ خطای بیت: صحت‌سنجی نمودار خطای بیت با `pkt_size` برابر با 10^5

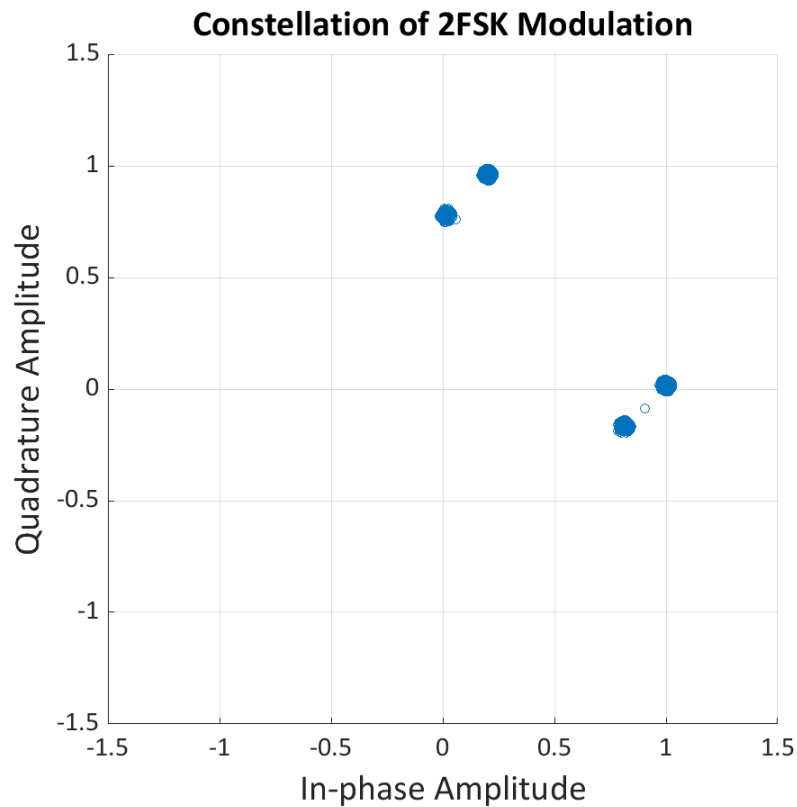
BER for 2FSK Modulation



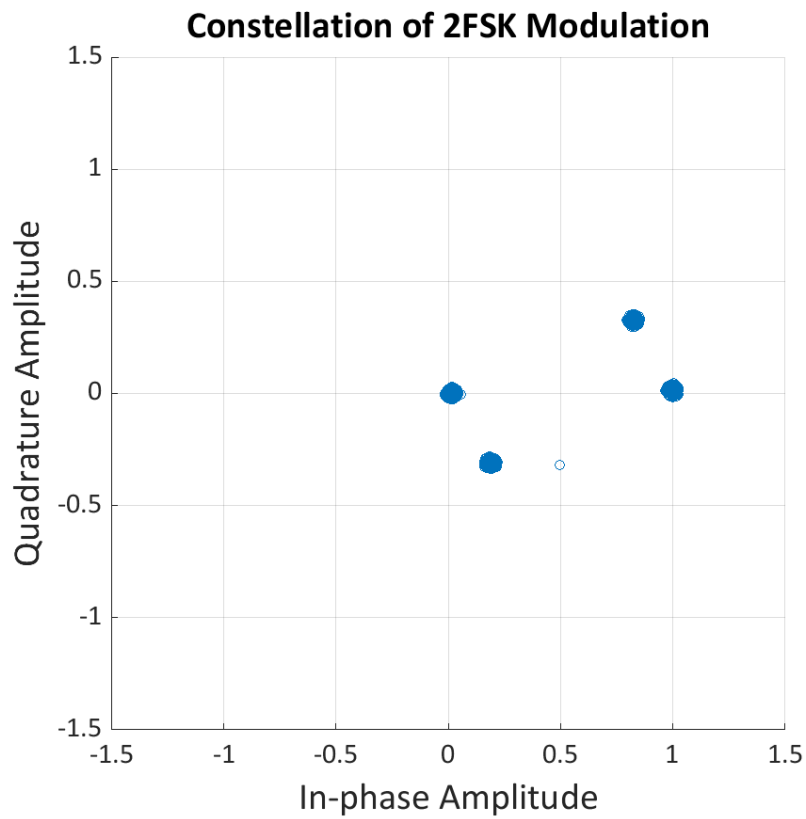
گام ۱۱. اثر تأخیر در کانال بر روی منظومه سیگنالی مدولاسیون 2FSK: منظومه‌ها با pkt_size برابر با 10^5 و نسبت سیگنال به نویز 40dB می‌باشد
منظومه با تأخیر صفر



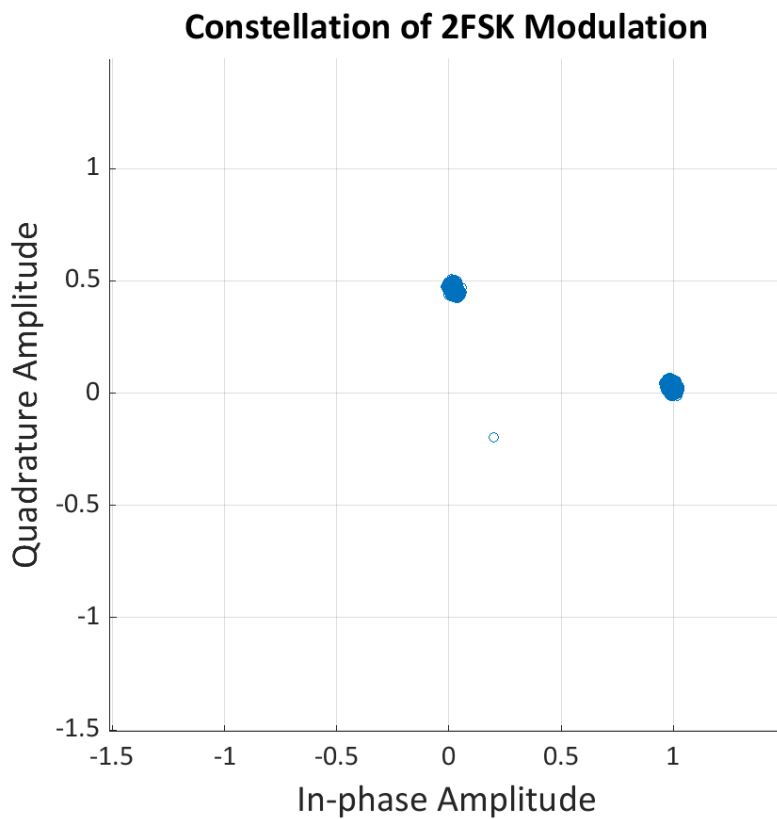
منظومه با تأخیر ۱/۰ طول سمبل



منظومه با تأخیر ۵/۰ طول سمبل



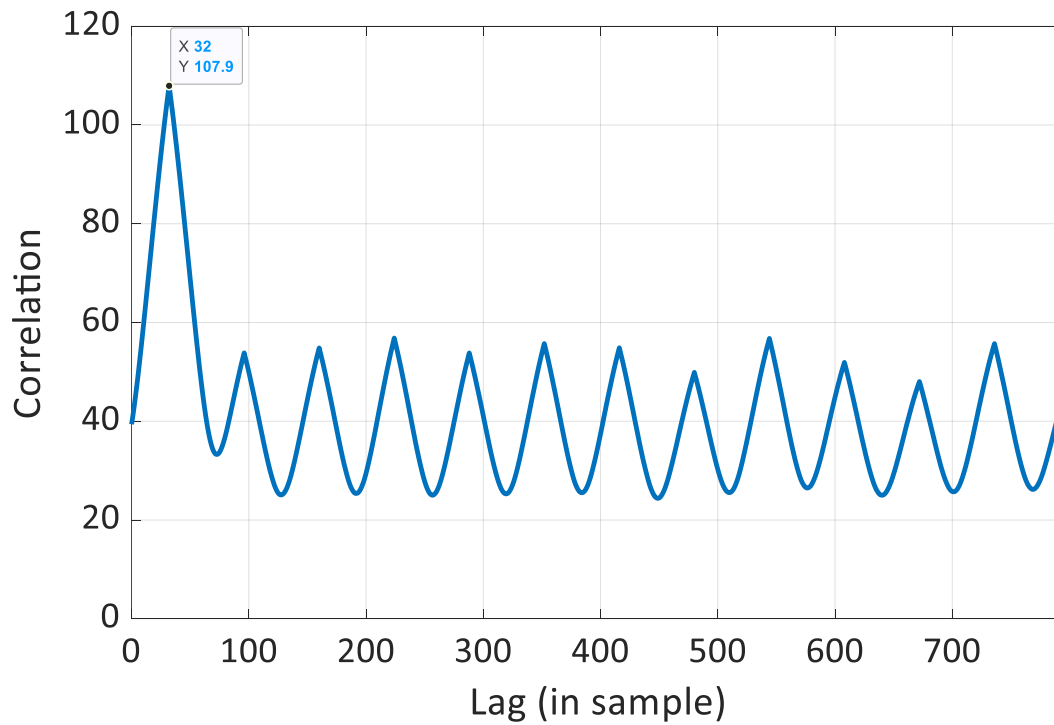
منظومه با تاخیر ۸/۰ طول سمبل



گام ۱۲. بیت‌های هدر:

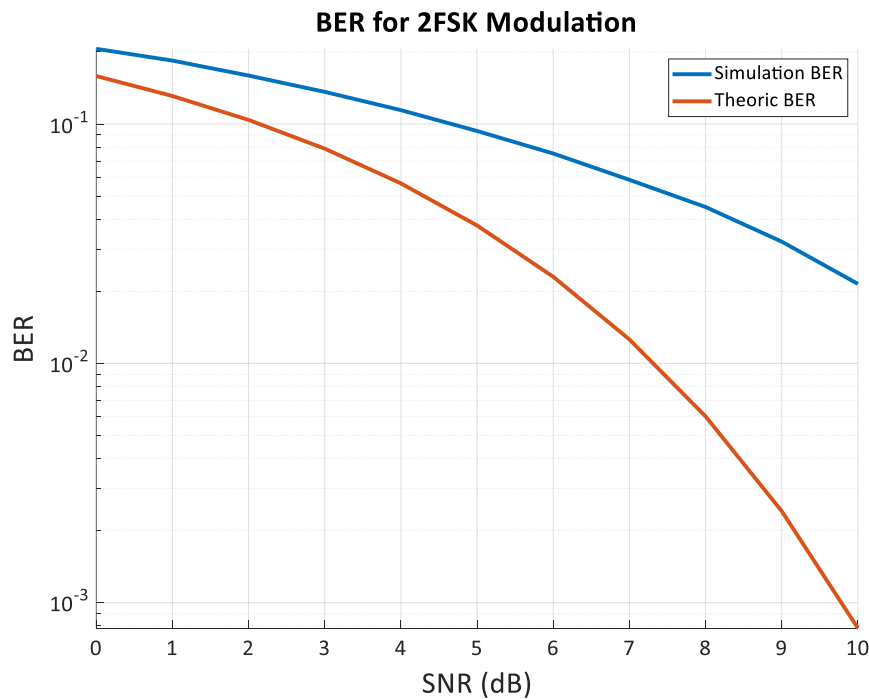
[1;0;0;1;0;1;1;1;1;0;0;1;0;1;0;1;0;0;1;1;1;1;0;1;0;1;0;0;1;1;0;1;1;1;0;0;1;0;1;
 ;1;1;1;0;0;0;0;0;0;0;0;0;0;0;1;1;0;0;1;1;1;1;0;1;1;0;0;0;1;1;0;0;1;0;1;0;1;1;
 ;1;0;1;0;0;1;0;1;1;0;1;1;0;1;1;0;0;0;1;1;1;0;1;0;1;1;1;0]

گام ۱۳. خروجی همبستگی با سیگنال هدر: بخشی از خروجی همبستگی برای تأخیر برابر با 0.5 طول سمبل

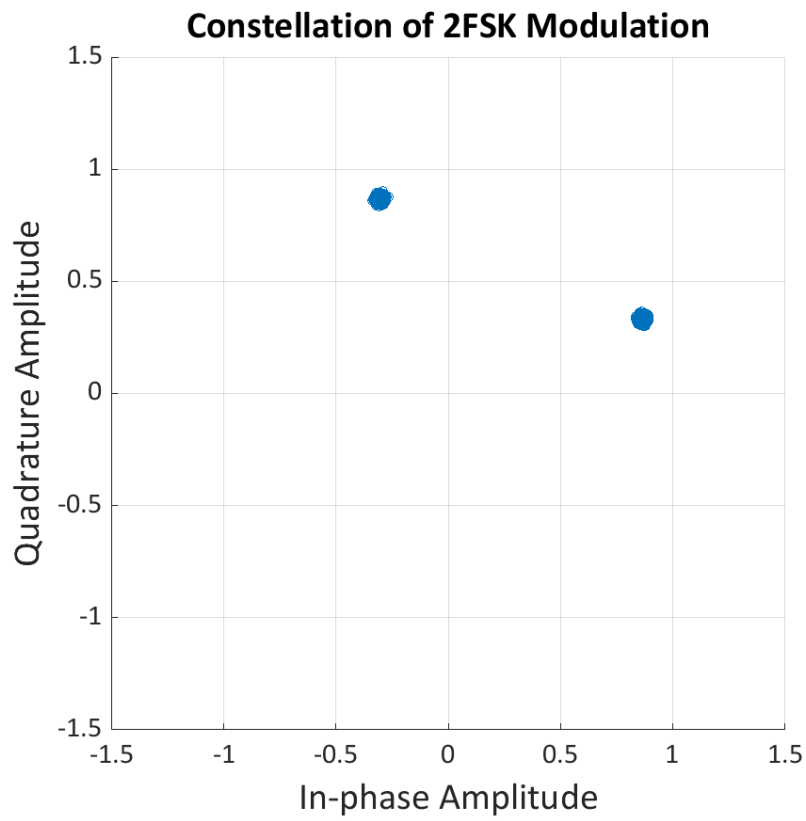


گام ۱۴. اطمینان از یک بودن نمودار نرخ خطای بیت بعد از جبران سازی تأخیر کانال

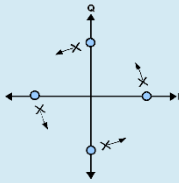
گام ۱۵. اثر فاز کانال بر روی نمودار نرخ خطای بیت: صحت سنجی نمودار خطای بیت با فاز کانال 30 درجه با `pkt_size` برابر با 10^5



گام ۱۶. اثر فاز کانال بر روی منظومه سیگنالی مدولاسیون 2FSK: منظومه‌ها با `pkt_size` برابر با 10^5 و نسبت سیگنال به نویز 40dB می‌باشد



گام ۱۷. اطمینان از یک بودن نمودار نرخ خطای بیت بعد از جبران سازی فاز کانال



آزمایش پنجم

آشکارسازی ناهمدوس

شرح آزمایش



آزمایش ۱-۵: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون FSK ناهمدوس و DBPSK آماده نماییم. از آن جا که در این آزمایش بناست این فرستنده با استفاده از رادیو نرم افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (f_s)، تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در بسته‌ی داده (pkt_size) و مدت زمان ارسال داده ($stop_time$) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت **Error! Reference source not found.** می‌باشد.

جدول ۱ پارامترهای آزمایش ۷

پارامتر	f_s	$nSymbolSamples$	pkt_size	$stop_time$
مقدار	10MHz	FSK: 64 DBPSK: 8	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

۱. تولید M-File مربوط به مقدار دهی اولیه: قالب کلی پارامترهای مورد نیاز در این آزمایش، مانند آزمایش قبل است و فایل `dcl_init.m` می‌بایست پارامتری به نام `mod_det_opt` داشته باشد که نوع آشکارسازی را که می‌تواند همدوس (`coherent`) یا ناهمدوس (`noncoherent`) باشد را مشخص می‌نماید. برنامه با توجه به این گزینه نوع گیرنده و نحوه‌ی ارسال فرستنده مدیریت می‌شود. هم‌چنین پارامتر `phase_amb_opt` برای نوع اعمال ابهام فاز در نظر گرفته شود. این فایل می‌بایست از برنامه‌ی اصلی فراخوانی نماید.

آزمایش ۲-۵: پیاده‌سازی مدولاسیون FSK ناهمدوس

۱. شبیه‌سازی کانال با فاز تصادفی: برای شبیه‌سازی کانال با فاز تصادفی می‌بایست بردار `txSamples_delayed` را در `exp(1i*chnl_phase_offset_amb)` ضرب شود. حاصل را در بردار `txSamples` قرار دهید. `chnl_phase_offset_amb` را به صورت یک بردار تصادفی بین ۰ و 2π تولید کنید. طول این بردار برابر با طول بردار `txSamples` می‌باشد. دقت کنید که فاز اضافه شده در طول هر سمبل ثابت باشد. در این حالت `phase_amb_opt` برابر با ۰ است. (راهنمایی: از دستور `kron` می‌توان برای این کار استفاده نمود).

۲. شبیه‌سازی مدولاسیون FSK ناهمدوس: برای پیاده‌سازی این مدولاسیون، همانند FSK همدوس که در آزمایش قبل پیاده شد عمل نمایید. نمودار احتمال خطای بیت را برای مدولاسیون 4FSK و 2FSK به دست آورده و نتیجه‌ی را با دستور `berawgn` صحت‌سنجی نمایید. دقت نمایید در حالت آشکارسازی ناهمدوس باید حداقل فاصله‌ی فرکانسی $1/T_s$ باشد. آشکارساز این مدولاسیون نیز به صورت $\arg \max |r_{\ell}, \underline{S}_{\ell,m}|$ می‌باشد.

۳. پیاده‌سازی سخت‌افزاری مدولاسیون FSK ناهمدوس: با فرض این که توان فرستنده‌ی ADALM-PLUTO برابر با 0dBm و بهره‌ی گیرنده برابر با 20dB باشد و از آنتن به منظور ارتباط فرستنده و گیرنده استفاده می‌شود، خطای آشکارسازی با استفاده از آشکارسازی ناهمدوس را برای مدولاسیون 2FSK محاسبه نمایید. هم‌چنین در این حالت منظومه‌ی سیگنالی این مدولاسیون را نیز رسم نمایید. در این جا از هدر تنها برای به دست آوردن ابتدای بسته‌ی ارسالی استفاده می‌شود و هیچ گونه جبران‌سازی فازی صورت نمی‌پذیرد.

آزمایش ۳-۵: پیاده‌سازی مدولاسیون DBPSK

۱. تولید بیت: با استفاده از تابع `bit_gen` تعدادی بیت ۰ و ۱ را متناسب با طول بسته‌ی داده و با در نظر گرفتن پارامترهای مدولاسیون DBPSK تولید نموده و آن را درون ماتریس `txBit` ذخیره نمایید. ماتریس `txBit` یک ماتریس با اندازه‌ی `pkt_size×1` می‌باشد.

۲. کد کردن بیت‌ها: با استفاده از **Error! Reference source not found.** رشته بیت‌های e_n را تولید نمایید و آن را در رون بردار `enc_txBit` قرار دهید.

۳. نگاشت بیت به سمبل:

گام ۱. کدگذاری گری بیت‌های تولیدی: با وجود این که در این جا نیازی به کدگذاری گری نیست ولی به منظور کلی بودن برنامه این مرحله نیز انجام می‌شود. با استفاده از تابع `grayMatrixGenerator`، ماتریس مربوط به کدگذاری گری مدولاسیون DBPSK را تولید کنید و آن را در ماتریس `grayMatrix` ذخیره کنید. بردار جدیدی متناظر با ماتریس `enc_txBit` و با نام `symbolIndex` تولید نمایید و نگاشتی یک‌به‌یک بین سطرهای ماتریس `enc_txBit` و ماتریس `grayMatrix` برقرار کنید. به عبارتی سطر نام بردار `symbolIndex`، شماره‌ی سطر از `grayMatrix` را نشان می‌دهد که برابر با سطر نام ماتریس `enc_txBit` باشد.

گام ۲. تولید سمبل‌های ارسالی: با استفاده از تابع `constellation`، تمامی سمبل‌های ارسالی مدولاسیون DBPSK را تولید کرده و سمبل‌های متناظر با هر سطر بردار `symbolIndex` را تولید کنید. این سمبل‌ها را در بردار `symbolArray` ذخیره نمایید.

۴. شکل‌دهی پالس ارسالی: با استفاده از تابع `pulseModulation` نمونه‌های ارسالی مربوط به یک بسته‌ی داده را آماده‌ی ارسال کرده و حاصل را در بردار `txSamples` ذخیره نمایید. تابع شکل‌دهی پالس را مثلی در نظر بگیرید.

آزمایش ۴-۵: مدل‌سازی کانال

۱. افزودن تأخیر در کانال: برای شبیه‌سازی تأخیر در کانال به اندازه‌ی پارامتر `chnl_delay_in_smp1` به ابتدای بردار `txSamples` صفر اضافه نمایید. حاصل را درون بردار `txSamples_delayed` قرار دهید. دقت نمایید که طول بردار افزایش می‌باید و می‌بایست در ادامه‌ی برنامه اثر این افزایش لحاظ شود. در این جا فعلاً مقدار تأخیر برابر با صفر قرار دهید.

۲. اعمال ابهام فاز کانال: مدولاسیون DPSK به تغییرات شدید فاز حساس است و بنابراین باید ابهام فاز ایجاد شده خیلی زیاد نباشد. ابهام فاز ایجاد شده برای سمبل‌ها را به صورت زیر مدل می‌کنیم

$$\text{chnl_phase_offset_amb} = 0:\text{phase_step}:(\text{pkt_size}-1)*\text{phase_step}$$

پارامتر **phase_step** مقدار ابهام فاز ایجاد شده برای یک سمبل نسبت به سمبل قبلی را مشخص می‌کند. حال با استفاده از دستور **kron**، فاز تولید شده را برای کل نمونه‌های هر سمبل تکرار کنید. با ضرب کردن عبارت $\exp(1j * \text{chnl_phase_offset_amb})$ در **txSamples_delayed**، ابهام فاز ایجاد شده را به سیگنال ارسالی اضافه نمایید و حاصل را در **txSamples** ذخیره نمایید. در این جا **phase_step** را برابر با ۱۰ درجه قرار دهید.

۳. شبیه‌سازی کانال با نویز سفید گاوسی

گام ۱. تعیین واریانس نویز بر اساس نسبت سیگنال به نویز (E_b/N_0): ابتدا بر اساس خروجی تابع **constellation**، مقدار متوسط انرژی سمبل را به دست آورید (**Es_avg**). حال با استفاده از متوسط انرژی سمبل، مقدار انرژی متوسط بیت را به دست آورده و برابر متغیر **Eb** قرار دهید. مقدار E_b/N_0 را برابر با 10dB در نظر بگیرید (به عبارتی پارامترهای **snr_min** و **snr_max** برابر 10 تنظیم می‌شوند). سپس واریانس نویز را بر اساس نسبت E_b/N_0 به دست آورید و آن را درون متغیر **var_noise** قرار دهید.

گام ۲. افزودن نویز به سیگنال: با استفاده از تابع **randn** یک بردار نویز مختلط با واریانس **var_noise** و ابعاد برابر با **txSamples** تولید نمایید و آن را **noise_smp1** نامگذاری نمایید. سپس این بردار را با بردار **txSamples** جمع نموده و حاصل را **txSamples_noise** بنامید.

آزمایش ۵-۵: پیاده‌سازی گیرنده‌ی مدولاسیون DBPSK

در ادامه‌ی فایل آزمایش قبل می‌خواهیم گیرنده‌ی مدولاسیون DBPSK را پیاده‌سازی نماییم.

۱. دمدولاسیون یا آشکارسازی پالس: نمونه‌های ارسالی شده به همراه نویز (**txSamples_noise**) را به عنوان نمونه‌های دریافتی درون بردار **rxSamples** قرار دهید. با استفاده از تابع **pulseDemodulation** با ورودی‌های نمونه‌های سیگنال دریافتی (**rxSamples**)، نام مدولاسیون (**modulation**)، مرتبه‌ی مدولاسیون (**M**)، نرخ نمونه‌برداری (**fs**)، تعداد نمونه‌های هر سمبل (**nSymbolSamples**)، نام تابع شکل‌دهنده‌ی پالس (**pulseName**)، روش تشخیص قدرت سیگنال دریافتی در راستای سیگنال‌های پایه (**mode**) و شیوه‌ی آشکارسازی (**det_opt**)، سمبل‌های دریافتی (**rxSymbols**) و اندیس سمبل‌های آشکار شده (**detectedSymbolsIndex**) را به دست آورید.

۲. محاسبه‌ی خطای سمبل: با مقایسه‌ی تعداد اختلاف‌های بردارهای **detectedSymbolsIndex** و **symbolIndex** و محاسبه نسبت اختلاف این دو بردار به تعداد کل سمبل‌ها، خطای سمبل را به دست آورده و آن را در متغیر **ser** قرار دهید.

۳. تبدیل سمبل و بیت و تبدیل از کدگذاری گری به کدگذاری باینری: بیت‌های متناظر با سطرهای **detectedSymbolsIndex** را به دست آورید و آن را درون متغیر **detectedBits_gray** قرار دهید. با وجود این که در این جا عکس تبدیل گری مطرح نیست به منظور کلی ماندن برنامه بیت‌های گری را به کدگذاری باینری تبدیل نماید و آن را درون بردار **detectedBits** قرار دهید.

۴. محاسبه‌ی خطای بیت: با مقایسه‌ی تعداد اختلاف ماتریس‌های **detectedBits** و **txBit** و محاسبه نسبت تعداد اختلاف این دو ماتریس به تعداد کل بیت‌ها، خطای بیت را به دست آورده و آن را در متغیر **ber** قرار دهید.

آزمایش ۵-۶: خواسته‌های کلی

۱. رسم نمودار نرخ خطای بیت: با تغییر نسبت سیگنال به نویز بین 0 تا 10dB نرخ خطای بیت را برای مدولاسیون‌های DBPSK به دست آورده و در یک نمودار رسم نمایید و آن را با خروجی تابع **berawgn** مقایسه نمایید. این کار را برای شکل موج مثلثی انجام دهید.

۲. اثر تغییرات فاز بر روی عملکرد: با تغییر پارامتر **phase_step**، تأثیر تغییرات فاز را بر روی عملکرد این سامانه ارزیابی نمایید.

آزمایش ۷-۵: پیاده‌سازی سخت‌افزاری مدولاسیون DBPSK

۱. ارسال نمونه‌ها با استفاده از ADALM-PLUTO: بردار `txSamples` مربوط به مدولاسیون DBPSK (هدر اضافه شود) را با استفاده از دستور `transmitRepeat` به مدت زمان `stop_time` ثانیه به صورت پی‌درپی در فضا ارسال نمایید. این دستور داده‌ها را از طریق USB به رادیو نرم‌افزار ارسال می‌نماید و در حافظه‌ی سخت‌افزار ذخیره می‌نماید. فرستنده داده‌ها را از حافظه‌ی رادیو نرم‌افزار قرائت کرده و مدام ارسال می‌نماید. برای این منظور ابتدا می‌بایست پس از پیکربندی رادیو نرم‌افزار یک شیء فرستنده ایجاد نموده و فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را برابر با 0dB تنظیم نمایید.

۲. دریافت سیگنال با استفاده از ADALM-PLUTO: ابتدا شیء مربوط به گیرنده را تعریف نمایید. فرکانس مرکزی آن را بر روی 2400MHz تنظیم کرده و بهره‌ی آن را به صورت Manual و برابر 20dB تنظیم نمایید. به علت اطمینان از دریافت یک بسته‌ی کامل، تعداد نمونه‌های زمانی دریافتی را برابر با دو برابر تعداد نمونه‌های زمانی بسته در نظر بگیرید. به منظور دریافت داده، شیء گیرنده را فراخوانی نمایید و حاصل را درون متغیر `rxSamples` قرار دهید.

۳. دمدولاسیون و مشاهده‌ی منظومه‌ی سیگنالی: مشابه قبل عمل دمدولاسیون را انجام داده و منظومه‌ی سیگنالی مربوط به `rxSymbols` را رسم نمایید و مشاهدات خود را یادداشت نمایید. منظومه‌ی سیگنالی باید بتواند به صورت به‌لحظه به‌روز شود.



[1] [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.



آزمایش ششم

انتقال دیجیتال از درون کانال باندمحدود AWGN



شرح آزمایش

آزمایش ۱-۶: مقداردهی‌های اولیه

در این آزمایش بنا داریم تعدادی بسته‌ی داده را برای ارسال از طریق مدولاسیون 2PAM آماده نماییم. از آن جا که در این آزمایش بناست این فرستنده با استفاده از رادیو نرم‌افزار ADALM-PLUTO پیاده‌سازی شود، می‌بایست با استفاده از پارامترها و ملاحظات عملی، این کار انجام شود.

پارامترهایی که در این پیاده‌سازی اهمیت دارد شامل نرخ نمونه‌برداری (fs)، تعداد نمونه‌های هر سمبل ($nSymbolSamples$)، تعداد سمبل‌های ارسالی در بسته‌ی داده (pkt_size) و مدت زمان ارسال داده ($stop_time$) می‌باشد. در این آزمایش مقدار پارامترهای مورد نیاز به صورت **Error! Reference source not found.** می‌باشد.

جدول ۲ پارامترهای آزمایش ۷

پارامتر	fs	$nSymbolSamples$	pkt_size	$stop_time$
مقدار	10MHz	32	شبیه‌سازی: ۱۰۰۰۰۰ سخت‌افزار: ۱۰۰۰	۱۰۰ ثانیه

دقت نظر داشته باشید که در این آزمایش نیز تنها می‌خواهیم برنامه‌هایی که در آزمایش قبل نوشته شده است را تکمیل نماییم.

آزمایش ۲-۶: پیاده‌سازی مدولاسیون 2PAM

۱. شبیه‌سازی مدولاسیون 2PAM: برای پیاده‌سازی این مدولاسیون، همانند آزمایش ۴ یا ۵ عمل نمایید. احتمال خطای بیت را برای نسبت $\frac{E_b}{N_0}$ برابر با 10dB به دست آورید. این عمل را برای دو شکل پالس مستطیلی و Root Raised Cosine ($\beta = 0.9$) و $span_in_syml = 6$ انجام دهید. (حالت عملکردی دمدولاتور را بر روی فیلتر منطبق تنظیم نمایید.)

۲. رسم نمودار چشمی: با استفاده از دستور **eyediagram** نرم‌افزار MATLAB نمودار چشمی مدولاسیون 2PAM را برای دو شکل پالس گفته شده و در دو نسبت E_b/N_0 برابر با 10dB و 40dB رسم نمایید. پارامترهای بر روی **Error! Reference source not found.** را به دست آورید.

آزمایش ۳-۶: مدل‌سازی کانال باندمباریک

۱. تولید یک فیلتر FIR: با استفاده از دستور **fir1** نرم‌افزار MATLAB یک فیلتر با پهنای باند 300kHz و با تعداد ۱۰۰ تپ تولید نمایید. پاسخ فرکانسی و پهنای باند نویز فیلتر را به دست آورید و با پهنای باند سیگنال ارسالی مقایسه نمایید.

۲. کانال باندمحدود: پس از اعمال تأخیر و اعمال ابهام فاز، سیگنال ارسالی حاصل را با استفاده از عمل کانولوشن از فیلتر FIR مرحله‌ی قبل عبور دهید. مجدد نمودار چشمی را در دو نسبت E_b/N_0 برابر با 10dB و 40dB رسم نمایید. احتمال خطای بیت را نیز به دست آورید. این عمل برای هر دو شکل پالس گفته شده انجام شود.

آزمایش ۴-۶: پیاده‌سازی کانال باندمحدود

۱. کار با ADALM-PLUTO: موارد گفته شده را با استفاده از رادیونرم افزار ADALM-PLUTO نیز انجام دهید.

آزمایش ۵-۶: خواسته‌های کلی

۱. اثر طول شکل پالس $\text{root raised cosine}$ با تغییر پارامتر span_in_syml تأثیر آن را بر روی نمودار چشمی و خطای بیت مدولاسیون 2PAM با شکل پالس $\text{root raised cosine}$ را مشاهده نمایید.



- [1] R. W. Stewart, K. Barlee, L. Crockett, and D. Atkinson, *Software Defined Radio using MATLAB & Simulink and the RTL-SDR*. 2015.